

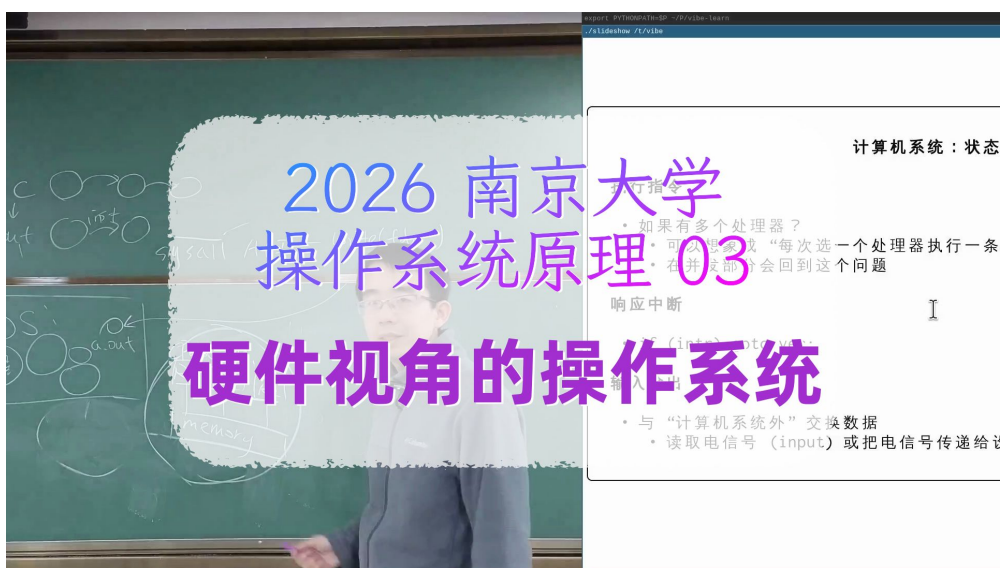
课程笔记

硬件视角的操作系统

南京大学 操作系统原理 2026 春季学期 第 03 讲

lecture-to-notes

2026 年 3 月 24 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 36 分钟

视频链接：<https://www.bilibili.com/video/BV18fcozAEsy>

目录

1 学习的方法	2
1.1 本章小结	3
2 硬件视角的操作系统	3
2.1 计算机系统的状态机模型	4
2.2 I/O 设备：状态机的“外部世界”	5
2.3 CPU Reset：初始状态	5
2.4 QEMU 调试：亲眼看 CPU Reset	6
2.5 本章小结	6
3 固件和 CIH 病毒	7
3.1 Firmware 的功能：配置计算机系统	7
3.2 Firmware 更新与安全隐患	8
3.3 CIH 病毒：操作系统安全的经典案例	8
3.4 本章小结	9
4 调试 Firmware	10
4.1 回到 43 年前：IBM PC-DOS 引导	10
4.2 用 QEMU 调试引导过程	11
4.3 本章小结	11
5 总结与延伸	12
5.1 讲者的核心信息	12
5.2 结构化提炼	12
5.3 跨章节关联	12
5.4 与前两讲的关联	12
5.5 开放问题	12
5.6 拓展阅读	13

1 学习的方法

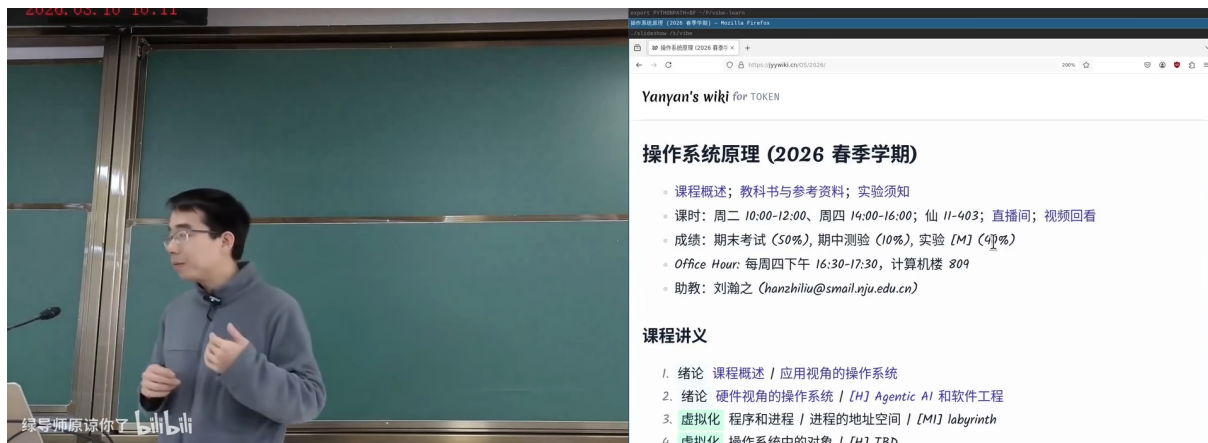


图 1: 课程 Wiki 与讲义结构¹

jyy 在正式进入硬件话题之前，用了一个完整的章节强调学习方法。核心观点：不要“干看”课程录像，要动手做。

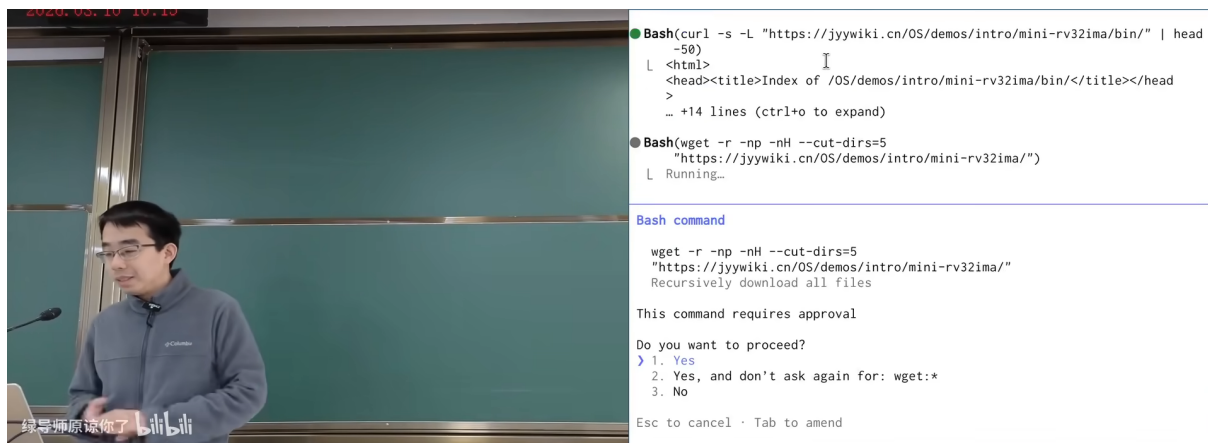


图 2: 现场演示：用 wget 下载课程 demo 代码²

jyy 现场演示了如何获取课程代码：

```
1 curl -s -L "https://jyywiki.cn/OS/demos/intro/mini-rv32ima/bin/" | head -50
2 wget -r -np -nH --cut-dirs=5 \
3     "https://jyywiki.cn/OS/demos/intro/mini-rv32ima/"
```

Listing 1: 下载课程 demo 代码

¹ 视频画面时间区间：00:01:00–00:01:30。

² 视频画面时间区间：00:04:45–00:05:15。

学习操作系统的方法

1. 获取代码：从课程 Wiki 下载所有 demo
2. 亲手运行：每个 demo 都自己编译运行一遍
3. 用 AI 辅助理解：不懂的代码让 Claude Code / Copilot 解释
4. 修改实验：改参数、加 `printf`、观察行为变化

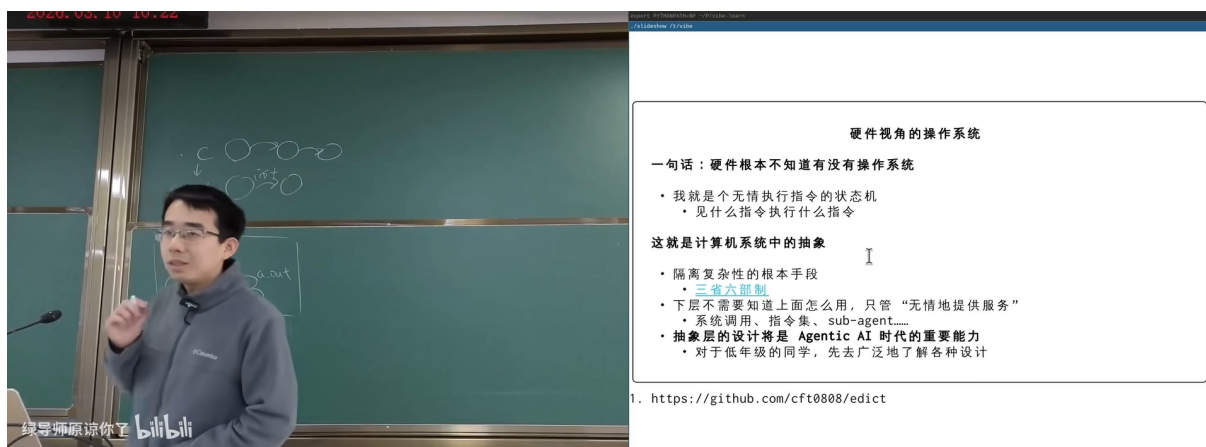
不要只是“看”课程——“做”才能真正理解。

1.1 本章小结

学习操作系统的核心方法：动手 > 看课 > 背概念。课程所有 demo 代码都开放在 Wiki 上，拿来跑就是最好的学习。

2 硬件视角的操作系统

这是本讲的核心章节。jyy 用一句话概括：**硬件根本不知道有没有操作系统。**



硬件视角的操作系统

一句话：硬件根本不知道有没有操作系统

- 我就是个无情执行指令的状态机
 - 见什么指令执行什么指令

这就是计算机系统抽象

- 隔离复杂性的根本手段
 - 三省六部制
- 下层不需要知道上面怎么用，只管“无情地提供服务”
 - 系统调用、指令集、sub-agent...
- 抽象层的设计将是 Agentic AI 时代的重要能力
 - 对于低年级的同学，先去广泛地了解各种设计

1. <https://github.com/cft0808/edict>

图 3：一句话：硬件根本不知道有没有操作系统³

³视频画面时间区间：00:09:30–00:10:15。

硬件视角的操作系统

一句话：硬件根本不知道有没有操作系统。

- 机器是个无情执行指令的状态机
- 它什么指令拿来就执行什么指令

这就是计算机系统中的**抽象**。

隔离复杂性的根本手段：

- 下层不需要知道上层如何实现服务
- 抽象层的设计界面是 Agentic AI 时代的重要能力
- 对于全球年轻人来说，也是一样的

2.1 计算机系统的状态机模型

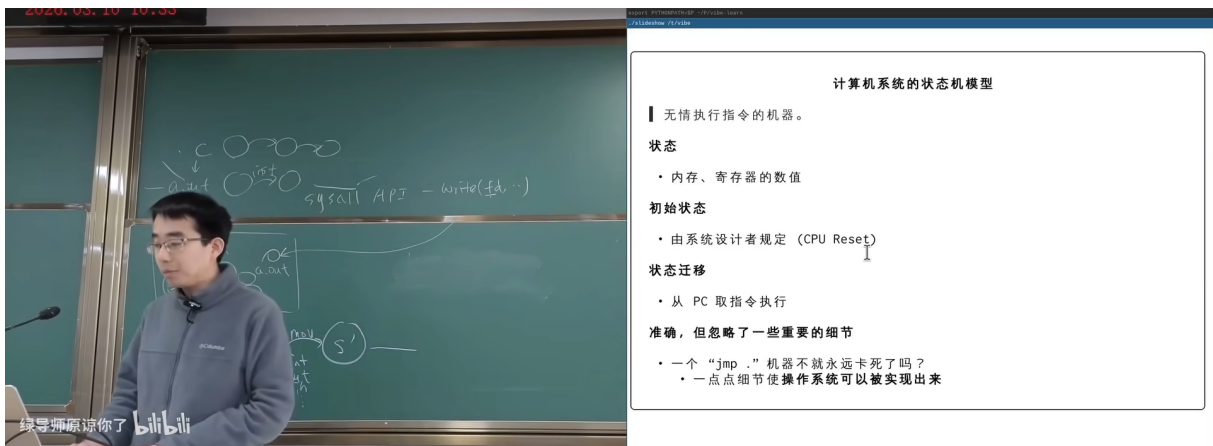


图 4: 计算机系统的状态机模型：状态、初始状态、状态迁移⁴

计算机系统

无情执行指令的机器：

状态：内存、寄存器的数值

初始状态：由系统设计者规定（CPU Reset）

状态迁移：从 PC 取指令执行

准确，但忽略了一些重要的细节：

- 一个“jmp .”——机器不就死了吗？
- 一点应用都不能做——操作系统可以做交互了吗？

⁴视频画面时间区间：00:18:00–00:18:45。

2.4 QEMU 调试：亲眼看 CPU Reset

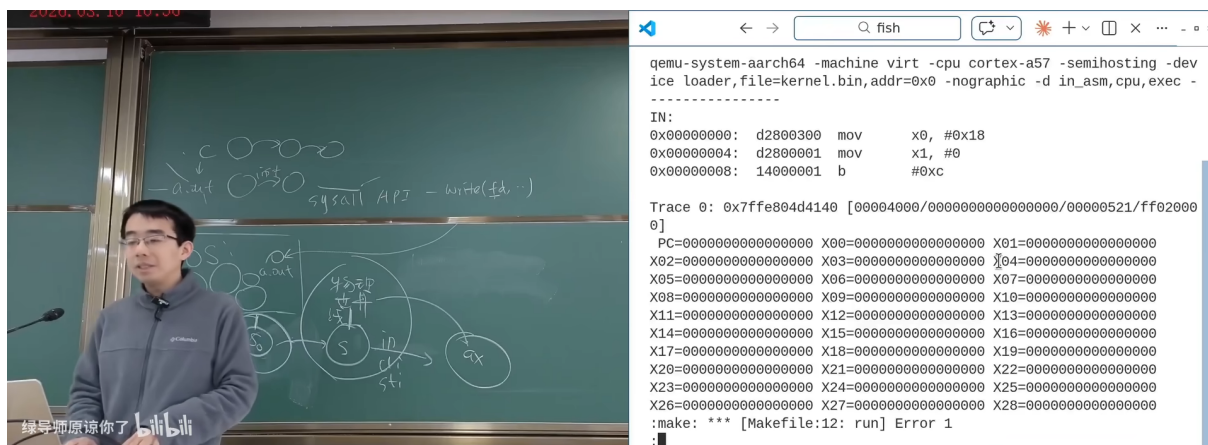


图 6: QEMU + GDB 调试：观察 CPU Reset 后的第一条指令⁶

jyy 用 QEMU 模拟 AArch64 系统，现场演示了 CPU Reset 后的执行：

```
1 qemu-system-aarch64 -machine virt -cpu cortex-a57 \  
2 -semihosting -device loader,file=kernel.bin,addr=0x0 \  
3 -nographic -d in_asm,cpu,exec
```

Listing 2: QEMU 启动 AArch64 裸机

可以看到 Reset 后的前几条指令：

```
1 0x00000000: d2800300 mov x0, #0x18  
2 0x00000004: d2800001 mov x1, #0  
3 0x00000008: 14000001 b #4
```

Listing 3: AArch64 Reset 后的指令

以及所有寄存器的初始状态——全部为 0 (PC=0, X0-X30=0)。

“在机器上调试”的重要性

很多同学只“看”代码，从不用调试器。但操作系统课最重要的技能之一就是**用 QEMU + GDB 单步执行**，亲眼观察每一条指令的效果。这比任何文档都直观——你看到的是真实 CPU 的行为，不是抽象描述。

2.5 本章小结

硬件不知道有没有操作系统——它只是无情地从 PC 取指令执行。CPU Reset 定义了确定的初始状态，PC 指向 Firmware 代码。I/O 设备是状态机与外部世界的接口。用 QEMU + GDB 可以亲眼观察这一切。

⁶视频画面时间区间：00:40:00-00:40:45。

3 固件和 CIH 病毒

3.1 Firmware 的功能：配置计算机系统

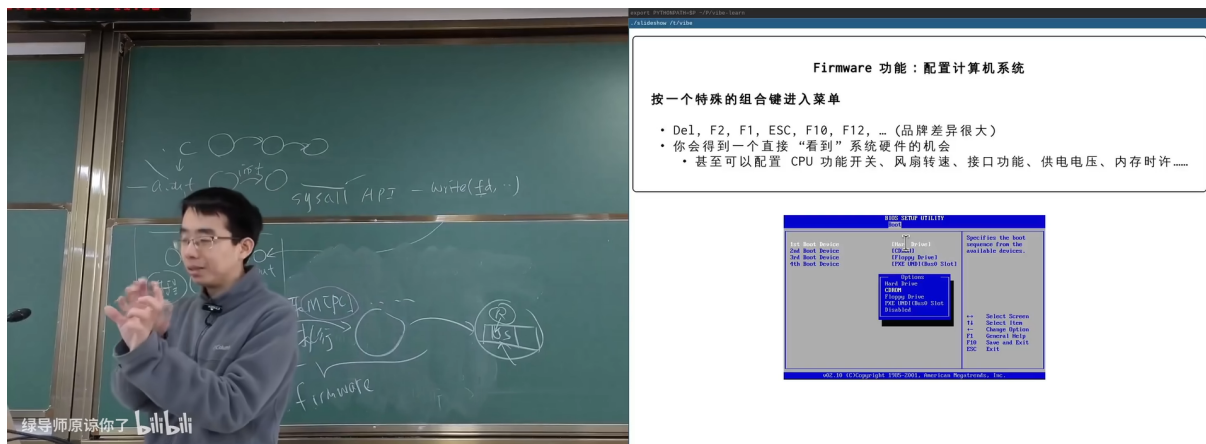


图 7: Firmware 功能：按特殊组合键进入配置菜单⁷

CPU Reset 之后执行的第一段代码就是 Firmware（固件）。Firmware 的职责：

- 按 Del/F2/F1/ESC/F9/F12..... 进入菜单（品牌差异很大）
- 自动检测一个叫做“重新刷”系统的东西软件的启动
- 甚至可以配置 CPU 功能开关、风扇转速、接口功能、供电电压、内存时序.....

BIOS vs. UEFI

BIOS (Basic Input/Output System): 传统固件，始于 IBM PC (1981)。16 位实模式，功能有限。

UEFI (Unified Extensible Firmware Interface): 现代固件标准。支持 64 位、图形界面、网络、安全启动 (Secure Boot)。本质上是一个小型操作系统。

两者的共同点：都是 CPU Reset 后第一个执行的软件，负责初始化硬件并加载操作系统。

⁷ 视频画面时间区间：00:58:00-00:58:45。

3.2 Firmware 更新与安全隐患

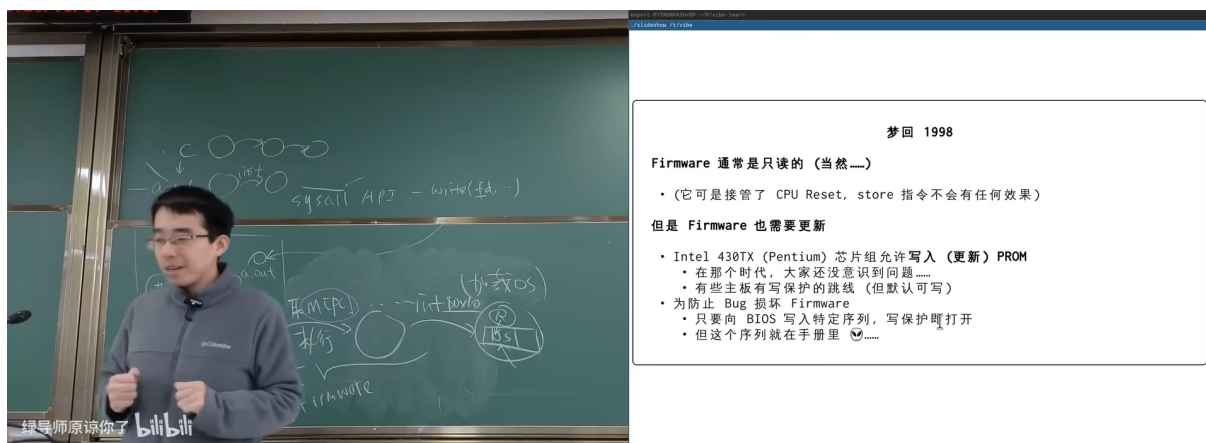


图 8: 梦回 1998: Firmware 更新的历史与安全问题⁸

jyy 回顾了 1998 年前后的 Firmware 安全问题：

- Firmware 通常是只读的——CPU Reset 后 store 指令不会有任何效果
- 但 Firmware 也需要更新——Intel 430TX (Pentium) 芯片组允许外部写入（更新）PROM
- 自那之后，大家都没想到什么问题……
- 为了修 Bug 就需要 Firmware 更新
- 打通了 BIOS 写入的大门，无法保护打开
- 但这有一个浮利就在于手里……

3.3 CIH 病毒：操作系统安全的经典案例

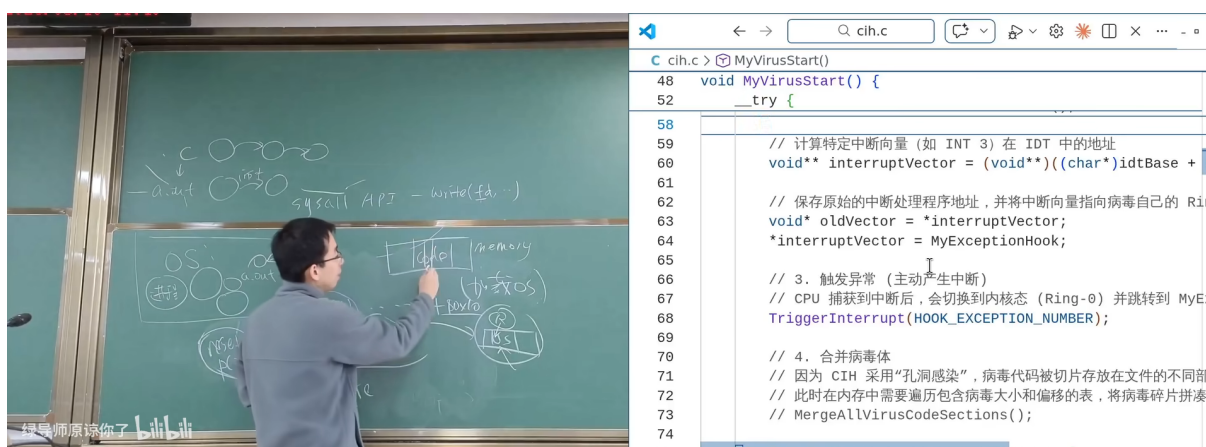


图 9: CIH 病毒源码分析：中断向量劫持⁹

jyy 展示了 CIH 病毒（1998）的实际源码，分析其工作原理：

⁸视频画面时间区间：01:06:00–01:06:45。

⁹视频画面时间区间：01:16:00–01:16:45。

```

1 void MyVirusStart() {
2     __try {
3         // 计算特定中断向量 (如 INT 3) 在 IDT 中的位置
4         void** interruptVector = (void**)((char*)idtBase + ...);
5         // 保存原始中断处理程序地址, 并替换为病毒代码
6         void* oldVector = *interruptVector;
7         *interruptVector = MyExceptionHook;
8         // 触发异常 (主动触发中断)
9         // 在 CPU 响应中断时, 会切换到 Ring-0 并跳转到 MyExceptionHook
10        TriggerException(HOOK_EXCEPTION_NUMBER);
11        // CIH 实现 "孔洞搜索": 病毒寻找可以填充的 PE 文件
12        // MergeAllVirusCodeSection()
13    }
14 }

```

Listing 4: CIH 病毒的核心: 中断向量劫持

CIH 的攻击链

1. 劫持 IDT (中断描述符表) 中的中断向量
2. 触发中断 → CPU 切换到 Ring-0 (内核态)
3. 在内核态下获得完全控制权
4. 写入 Firmware (BIOS) → 物理破坏 (板子变砖)
5. 覆盖硬盘 MBR → 数据丢失

CIH 是历史上第一个能物理损坏硬件的病毒——因为它利用了 Firmware 可写入的安全漏洞。

CIH 的启示

CIH 告诉我们: 操作系统安全不仅仅是“不让恶意程序运行”。如果硬件接口 (如 Firmware 写入) 没有被正确保护, 任何获得内核态权限的代码都可以造成永久性破坏。这就是为什么现代系统有 Secure Boot、TPM 等硬件级安全机制。

3.4 本章小结

Firmware 是 CPU Reset 后第一个执行的代码, 负责初始化硬件并加载 OS。1998 年的 CIH 病毒利用了 Firmware 可写入的漏洞, 实现了从中断劫持到物理破坏的完整攻击链。这个案例深刻说明了硬件-软件安全边界的重要性。

4 调试 Firmware

4.1 回到 43 年前：IBM PC-DOS 引导

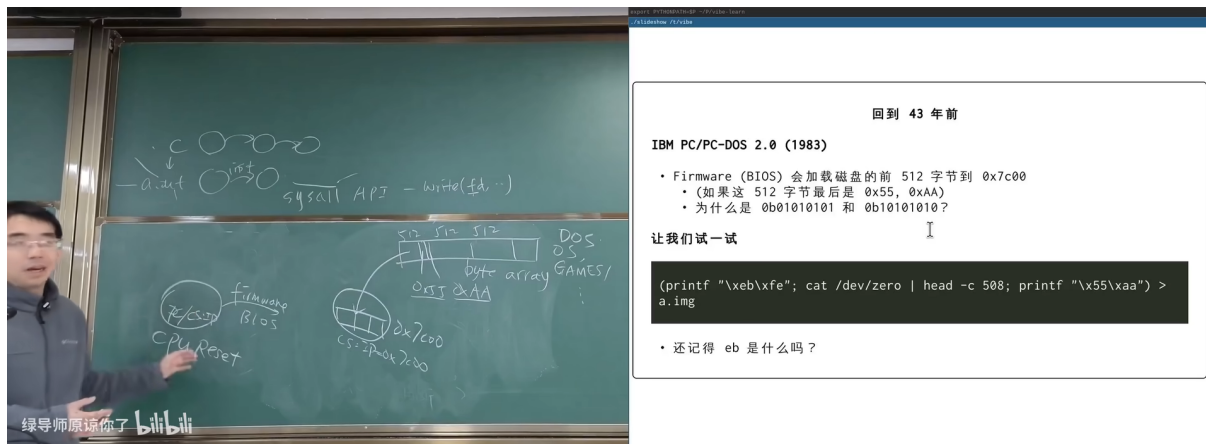


图 10: IBM PC/PC-DOS 2.0 (1983): Firmware 从磁盘加载 512 字节到 0x7c00¹⁰

jyy 带大家回到 1983 年的 IBM PC/PC-DOS 2.0 时代：

- Firmware (BIOS) 会把磁盘的前 **512 字节**读到内存地址 0x7c00
- 如果这 512 字节的最后两个字节是 0x55, 0xAA——就是一个合法的引导扇区
- 然后 CPU 跳转到 0x7c00 开始执行

```
1 (printf "\xeb\xfe"; cat /dev/zero | head -c 508; printf "\x55\xaa") > a.img
```

Listing 5: 构造一个最小的引导扇区

eb fe 是什么码？

0xEB 0xFE 是 x86 的“jmp .”（无限循环跳转到自身）。这是最简单的“可引导程序”——虽然什么都不做，但 BIOS 会认为它是合法的 boot sector 并跳转执行。

¹⁰ 视频画面时间区间：01:25:00–01:25:45。

4.2 用 QEMU 调试引导过程

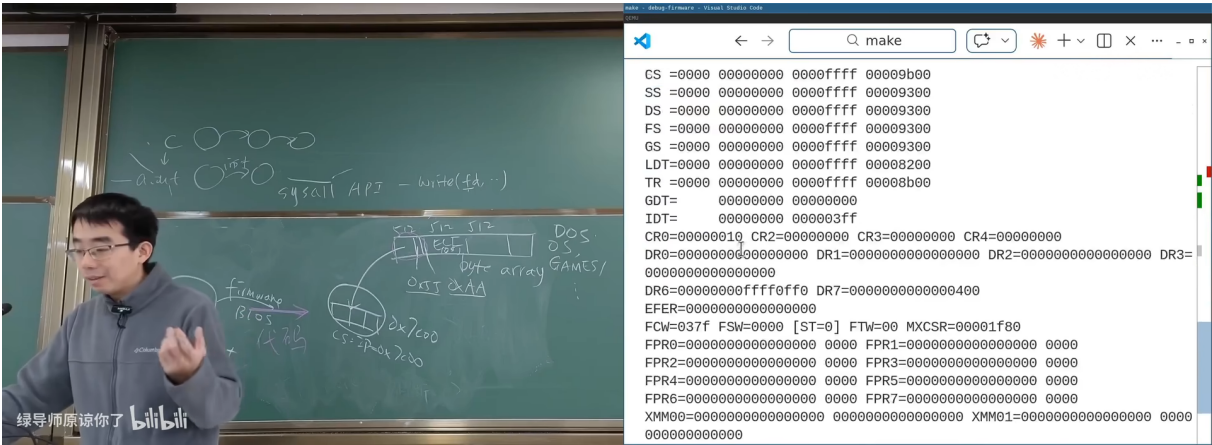


图 11: QEMU 调试：观察引导时 CPU 的所有寄存器状态¹¹

jyy 用 QEMU 启动刚构造的 a.img，通过 info registers 观察 CPU 状态：

```
1 CS =f000 00000000 0000ffff 00009300
2 DS =0000 00000000 0000ffff 00009300
3 GDT=      00000000 0000003ff
4 IDT=      00000000 000003ff
5 CR0=00000010 CR2=00000000 CR3=00000000 CR4=00000000
```

Listing 6: QEMU Monitor 中的寄存器状态

可以清楚看到：

- CS=0xF000：代码段寄存器指向高地址 ROM 区域
- GDT/IDT 都指向 0——还没有设置保护模式
- CR0 的 PE 位为 0——CPU 在实模式（Real Mode）

从 Real Mode 到 Protected Mode

x86 CPU Reset 后处于 16 位实模式（Real Mode），寻址空间仅 1MB。操作系统引导的关键步骤之一就是设置 GDT、切换到保护模式（Protected Mode）/长模式（Long Mode），获得完整的 64 位寻址空间和内存保护能力。这就是为什么理解 Firmware 和引导过程如此重要——它是“从硬件到操作系统”的桥梁。

4.3 本章小结

IBM PC 的引导协议简单而优雅：BIOS 读取磁盘前 512 字节到 0x7c00，检查魔数 0x55AA，然后跳转执行。用 QEMU + GDB 可以在 CPU 寄存器级别观察整个引导过程——从 Real Mode 到操作系统接管。

¹¹ 视频画面时间区间：01:30:00–01:30:45。

5 总结与延伸

5.1 讲者的核心信息

第三讲建立了一个从“裸硬件”到“操作系统加载”的完整链条：

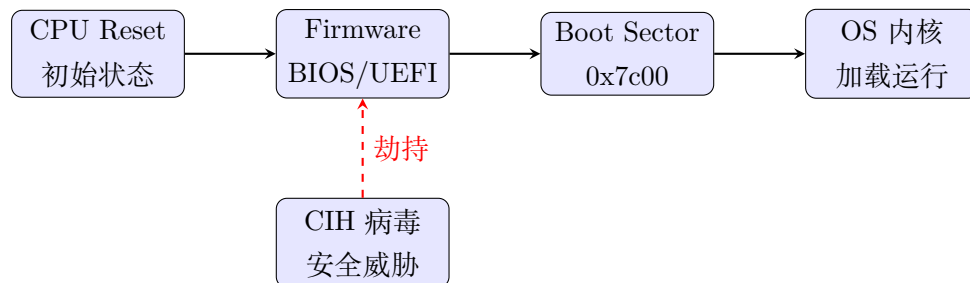
CPU Reset $\xrightarrow{\text{PC 指向 ROM}}$ Firmware $\xrightarrow{\text{读取 Boot Sector}}$ Boot Loader $\xrightarrow{\text{加载内核}}$ OS

5.2 结构化提炼

四个核心 Takeaway

1. **硬件不知道有没有 OS**——CPU 只是无情地取指令执行。操作系统是“软件假装的”抽象层，硬件不关心上面跑的是 OS 还是恶意代码。
2. **CPU Reset 定义一切的起点**——所有寄存器清零，PC 指向固定地址。这个地址放的就是 Firmware，是引导链的第一环。
3. **Firmware 是 OS 之前的 OS**——初始化硬件、检测设备、配置参数、加载引导扇区。BIOS → UEFI 的演化反映了硬件复杂性的增长。
4. **CIH 证明了安全边界的重要性**——如果 Firmware 可写入且不受保护，任何获得内核权限的代码都能物理破坏硬件。Secure Boot、TPM 就是后来的补救。

5.3 跨章节关联



5.4 与前两讲的关联

- 第 01 讲说“操作系统是管理硬件和软件的软件”→ 第 03 讲揭示了硬件根本不知道 OS 的存在
- 第 02 讲说“程序 = 状态机，syscall = OS 接口”→ 第 03 讲说明了在 syscall 之前，CPU 经历了 Reset → Firmware → Boot → OS 的完整启动链
- 第 02 讲用 strace 观测程序行为 → 第 03 讲用 QEMU + GDB 观测 CPU 寄存器级行为

5.5 开放问题

1. UEFI Secure Boot 如何防止类似 CIH 的攻击？
2. ARM 架构（如手机）的引导链和 x86 有什么不同？
3. 在 Agentic AI 时代，Firmware 级别的安全威胁是否会更严重？

5.6 拓展阅读

- jyy 的课程 Wiki: <https://jyywiki.cn/>
- CIH 病毒源码分析: <https://github.com/cft8888/cih>
- UEFI 规范: <https://uefi.org/specifications>